
CacheEdit: Efficient Multi-round Image Editing via Adaptive Token-wise Reuse

Jinxin Yu^{1,2} Xueqing Cheng^{2,3,4} Yudong Pan^{1,2} Lian Liu^{1,2} Shengwen Liang^{1,2} Huawei Li^{1,2} Xiaowei Li^{1,2}
Ying Wang^{1,2}

Abstract

Instruction-based image editing (IIE) is a vital tool for iterative content creation, enabling multi-round interactions that refine visual details while preserving cross-round consistency. However, this workflow is constrained by the compute-bound nature of Diffusion Transformers (DiTs): because DiTs process tokens uniformly, they waste substantial computation on regions untouched by the instruction. We investigate the Round–Step–Layer hierarchy of DiT-based editing and identify a phenomenon we term Delayed Latent Emergence (DLE). Although pronounced latent changes emerge only in the late denoising stages, deep-layer activations within transformer blocks at the very first sampling step already diverge markedly in edited regions. Building on this insight, we propose CacheEdit, a training-free framework centered on an Adaptive Activation Cache (Acache) that exploits early-step sensitivity to detect invariant tokens and reuse their cached activations across subsequent sampling steps, thereby bypassing redundant computation. Experiments on FLUX.1 Kontext and Qwen-Image-Edit show that CacheEdit achieves up to $2.5\times$ end-to-end acceleration. Moreover, by isolating and reusing static features, CacheEdit mitigates stochastic drift and improves instruction-following and structural consistency over full-recomputation baselines.

1. Introduction

Diffusion models (Rombach et al., 2022; Croitoru et al., 2023) have demonstrated high efficacy across various generative tasks, particularly in high-fidelity image synthesis and content creation (Cao et al., 2025). Building on these advancements, instruction-based image editing (IIE) (Kawar et al., 2023; Huang et al., 2025) has emerged as a primary method for interactive manipulation, enabling users to perform complex visual modifications through natural language instructions. Modern architectures such as Flux.1 Kontext (Batifol et al., 2025) and Qwen-Image-Edit (Wu et al., 2025) leverage large-scale Diffusion Transformers (DiTs) to achieve precise semantic alignment. However, while these models offer superior performance, their deployment in multi-round interactive scenarios is constrained by significant computational requirements. This burden results in high inference latency that severely compromises the interactive user experience, especially as users often perform multiple iterative refinements on a single image.

Beyond the inherent compute intensity of DiTs (Peebles & Xie, 2023), image editing exposes an additional inefficiency stemming from their uniform execution paradigm. Despite user instructions typically targeting only localized regions, the model processes the entire image with the same computational complexity, applying full Transformer blocks, including self-attention and feed-forward networks to all tokens. This indiscriminate processing leads to significant computational redundancy in static regions that remain unchanged from previous iterations. Hence, exploring the redundant characteristics of non-edited regions and establishing methods for the early-stage identification of edited/unedited areas throughout the denoising trajectory become crucial prerequisites for achieving efficient inference.

Conventional efforts to accelerate diffusion models have predominantly centered on the task of standard image generation. Specifically, model pruning (Castells et al., 2024; Rombach et al., 2022) and quantization (Li et al., 2023; So et al., 2023) have been widely adopted to reduce model size and bit-widths, respectively. Knowledge distillation has also been extensively investigated to achieve both structural compression (Kim et al., 2023) and a reduction in inference trajectories (Sauer et al., 2024). Beyond these, contempo-

¹SKLP, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China ²University of Chinese Academy of Sciences, Beijing, China ³Computer Network Information Center, Chinese Academy of Sciences, Beijing, China ⁴Zhongguancun Academy, Beijing, China. Correspondence to: Huawei Li <li-huawei@ict.ac.cn>, Xiaowei Li <lxw@ict.ac.cn>, Ying Wang <wangying2009@ict.ac.cn>.

rary works (Wimbauer et al., 2024; Ma et al., 2024a;b) capitalize on the temporal redundancy of activations, utilizing similarity-based cache reuse to further minimize computational costs. Nonetheless, the redundancy characteristics and the feasibility of efficient inference in the burgeoning multi-round IIE framework have not been sufficiently addressed. While RegionE (Chen et al., 2025b) leverages step-wise latent variations to identify static regions for KV-cache reuse, its scope is limited to a single round, failing to exploit inter-round redundancy, and it often cannot effectively isolate edited regions until the later sampling steps. On the other hand, InstGenIE (Jiang et al., 2025) supports inter-round activation sharing but relies on user-provided masks, which makes it unsuitable for general IIE scenarios where the boundaries between edited and unedited regions are not explicitly defined.

We conduct a comprehensive investigation into the internal behavior of DiTs during multi-round editing, analyzing the similarity of various high-dimensional image representations across a three-tier hierarchy: round, step, and layer. Specifically, the round level entails comparisons between two complete image editing processes; the step level examines the progression across multiple denoising steps within a single editing iteration; and the layer level refers to the multiple Transformer blocks within an individual denoising step. We identify a pivotal phenomenon termed Delayed Latent Emergence (DLE). As shown in Figure 1, latent-space differences between consecutive rounds remain small early in the denoising trajectory and become pronounced only in the mid-to-late stages. Consequently, latent-based similarity is insufficient for identifying edited regions during early sampling. In contrast, we observe a pronounced Early Layer Sensitivity: even at the initial denoising step, output activations from deep Transformer blocks manifest substantial variance in edited areas. These findings suggest that internal model features possess the capacity to perceive and encode editing intent far ahead of the terminal latent representations.

Building on these observations, we propose CacheEdit, a training-free inference framework centered on an Adaptive Activation Cache (Acache). At the first denoising step, CacheEdit identifies unedited tokens by comparing deep-block activations with those from the previous round. For subsequent steps, it reuses cached activations for these tokens, bypassing redundant computation in the Transformer blocks. By decoupling edited (dynamic) from unedited (static) regions, CacheEdit reduces compute and delivers substantial speedups, achieving up to $2.5\times$ end-to-end speedup. Beyond improving efficiency, CacheEdit also enhances editing quality, as evidenced by VLM-based metrics: on FLUX.1 Kontext, the GPT-4o overall score (G-O) increases from 6.336 to 6.814, indicating better adherence to instructions and improved cross-round consistency.

The primary contributions of this work are as follows:

- **Observation of Delayed Latent Emergence:** We provide a systematic analysis of cross-round redundancy and demonstrate that deep-layer activations are more sensitive indicators for identifying edited regions in early steps than image latents.
- **Acache Mechanism:** We propose a training-free caching framework that adaptively reuses token-wise activations across rounds, transitioning DiT inference from a uniform process to a demand-aware paradigm.
- **System Efficiency:** Experimental results show that CacheEdit achieves up to $2.5\times$ end-to-end acceleration on state-of-the-art open-source models such as Flux.1 Kontext and Qwen-image-edit.
- **Enhanced Consistency and Instruction Following:** By explicitly preserving static features, our method improves background stability and yields superior instruction-following performance compared to standard re-computation baselines.

2. Background & Related Work

2.1. Instruction-based image editing

Instruction-based image editing (IIE) (Kawar et al., 2023) extends text-to-image generation by conditioning the generative process on both an input image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$ and a natural-language instruction $\mathbf{y}$. Unlike pure text-to-image synthesis, the model learns a mapping $\mathcal{F} : (\mathbf{x}, \mathbf{y}) \rightarrow \mathbf{x}'$ that produces an edited image $\mathbf{x}'$ while balancing two objectives: (i) faithfully applying the semantic modification specified by $\mathbf{y}$ (for example, changing objects, attributes, or style), and (ii) preserving the content in $\mathbf{x}$ that is irrelevant to the instruction.

Execution Flow. In practice, modern methods typically operate within the latent space of a pre-trained Variational Autoencoder (VAE). The execution flow proceeds as follows:

1. **Encoding:** The input image $\mathbf{x}$ is first encoded into a low-dimensional latent representation $\mathbf{z} = \mathcal{E}(\mathbf{x})$ via the VAE encoder $\mathcal{E}$, while the instruction $\mathbf{y}$ is mapped to a sequence of textual embeddings $\mathbf{c} = \Gamma(\mathbf{y})$ (e.g., via CLIP (Radford et al., 2021)).
2. **Conditional Denoising:** The generation is modeled as a reverse diffusion chain starting from Gaussian noise $\mathbf{z}_T$. At each timestep t , the backbone ϵ_θ predicts noise conditioned on the current noisy state $\mathbf{z}_t$, the instruction embedding $\mathbf{c}$, and the visual condition $\mathbf{z}$

(often injected via concatenation or cross-attention):

$$\mathbf{z}_{t-1} \leftarrow \text{Sampler}(\mathbf{z}_t, \epsilon_\theta(\mathbf{z}_t, t, \mathbf{c}, \mathbf{z})) \quad (1)$$

Here, the visual condition $\mathbf{z}$ acts as a structural anchor, ensuring the layout and identity are preserved.

3. **Decoding:** Finally, the edited latent $\mathbf{z}'_0$ obtained at the end of the denoising path is decoded back to pixel space via the VAE decoder: $\mathbf{x}' = \mathcal{D}(\mathbf{z}'_0)$.

Because the output $\mathbf{x}'$ shares the same domain as $\mathbf{x}$, it can be fed back as the next context image, naturally supporting multi-round, iterative editing workflows.

Compared to standard text-to-image models, which generate an image from text alone and may freely invent visual content consistent with the prompt, instruction-based editors transform a *given* image into a *modified* one under language guidance. This introduces strong constraints on preservation and consistency, since the goal is not “generate an image that matches this description,” but instead “edit this image according to this description while keeping everything else unchanged.” As illustrated in Section B, this unified formulation supports local and global edits, identity-sensitive edits, style transfer, and in-context personalization within a single model.

2.2. Accelerating diffusion models

A key limitation of diffusion models is their high sampling cost. Standard DDPM-style samplers (Ho et al., 2020) often require tens to hundreds of denoising steps, and each step invokes a large neural network. This has motivated extensive work on faster sampling for generation.

General acceleration techniques. While relatively few methods are tailored specifically to IIE, a broad set of techniques have been proposed for diffusion models in general. From the perspective of parameter and numerical redundancy, researchers have introduced pruning methods such as Mosaicdiff (Guo et al., 2025), LD-Pruner (Castells et al., 2024), and BaWA (Liu et al., 2025d), quantization methods such as Q-diffusion (Li et al., 2023), FPQuant (Zhao et al., 2025), Q-DiT (Chen et al., 2025a), linear symmetric quantization (Zhao et al., 2020), Drift (Liu et al., 2024), and architecture-quantization co-optimization (Liu et al., 2023). Knowledge distillation has also been extensively investigated such as Swiftbrush (Nguyen & Tran, 2024) and Dkdm (Xiang et al., 2025). In addition, sparse attention mechanisms such as DiTFastAttn (Yuan et al., 2024), FlashOmni (Qiao et al., 2025), and Re-ttention (Chen et al., 2025c) aim to reduce the computational cost of attention.

Temporal and spatial redundancy. For temporal redundancy, several works such as DeepCache (Ma et al., 2024b), dkv-cache (Ma et al., 2025), OmniCache (Chu et al., 2025),

and TeaCache (Liu et al., 2025a) exploit similarity between adjacent diffusion steps to reuse intermediate features. For spatial redundancy, ToCa (Zou et al., 2024a) and DuCa (Zou et al., 2024b) observe that token sensitivity varies across the spatial grid, and they accelerate inference by updating only a subset of critical tokens at each timestep. Related cache- and memory-aware ideas have also been explored in Transformer serving systems, where locality-aware decoding, low-bit activation/KV-cache quantization, and memory-hierarchy-aware offloading are used to reduce repeated computation or data movement (Liu et al., 2025b;c).

IIE acceleration. Instruction-based editing is characterized by localized modifications and iterative, multi-round workflows. These features are not fully addressed by existing optimization strategies, which are primarily designed for one-shot global generation and therefore do not exploit the redundancy present in localized interactive editing. Only a few recent studies explore acceleration strategies tailored to IIE. RegionE (Chen et al., 2025b) uses step-wise latent variations to identify static areas for KV-cache reuse, but it is limited to single-round processing and does not exploit inter-round redundancy. Moreover, RegionE often struggles to isolate non-edited regions until later sampling stages, because latent representations evolve slowly at early timesteps, a limitation we refer to as Delayed Latent Emergence. InstGenIE (Jiang et al., 2025) enables inter-round activation sharing but requires explicit user masks, which makes it unsuitable for general instruction-only settings where edited regions are not provided.

3. Preliminary Study

3.1. Internal Similarity Analysis.

We conduct a systematic study of the internal dynamics of Diffusion Transformers in multi-round instruction-based image editing. We analyze cross-round similarity for several internal representations, including latent states, hidden activations, and Key-Value (KV) caches, and relate their variation patterns across rounds, sampling steps, and transformer layers to the underlying editing intent.

As shown in the left panel of Figure 1, image latents provide a high-dimensional representation that is closest to the pixel domain and is updated at each denoising step using the predicted noise. Many prior inter-step caching approaches rely on latent-level changes to decide whether to reuse computation. However, latent differences in edited regions become evident only in the mid-to-late denoising stages. As a result, latent-based similarity misses substantial redundancy that exists in the early, computationally intensive steps.

To overcome this limitation, we examine deeper internal variables. The right panel of Figure 1 reports cross-round similarity of output activations from each Transformer block

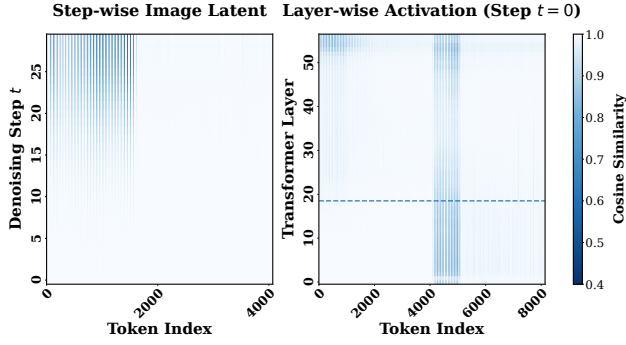


Figure 1. Similarity evolution of different image representations between consecutive rounds in an IIE model. The left plot shows the similarity of image latents across denoising steps, where substantial token-level divergence appears mainly in the mid-to-late stages. The right plot shows layer-wise similarity of output activations at the initial step ($t = 0$). Compared with latents, activations in deeper layers shift noticeably even at the first step.

at the initial sampling step ($t = 0$). Tokens are arranged so that the first half corresponds to the generated image and the second half corresponds to the reference image. We find that the model localizes edited regions in the reference tokens already in early layers. By the final layers of the first step, the corresponding regions in the generated image tokens also show clear divergence. Recording the indices of these highly varying tokens allows us to separate edited and non-edited regions at the start of denoising. Additional detail analysis is provided in Section C.

3.2. Key Insight

The similarity analysis suggests that intermediate activations in non-edited regions remain nearly invariant across rounds. However, DiTs process all tokens with roughly uniform compute, which leads to substantial redundancy. In interactive multi-round editing, where responsiveness is essential, this redundancy becomes a key bottleneck. Based on the observed behavior of different representations, we summarize two insights:

Observation 1: Delayed Latent Emergence (DLE). Changes in the image latent space evolve slowly and become clearly observable mainly in the mid-to-late denoising steps. This behavior is consistent with diffusion sampling, where the predicted noise is scaled as a function of the timestep, so later steps have a stronger influence on the final output.

Observation 2: Layer sensitivity in early steps. Within the Transformer, the instruction is incorporated rapidly, and deep-layer activations can change substantially even at the first sampling step. Although timestep-dependent scaling

can delay visible changes in the global latents, these internal activation differences provide a reliable signal for localizing edited regions. Leveraging this early sensitivity enables token-level separation of edited and non-edited regions, which allows redundant computation in static areas to be eliminated from the beginning of denoising.

4. Methods

4.1. System Overview

As illustrated in Figure 2, CacheEdit accelerates multi-round instruction-based editing by reusing computations for regions that remain unchanged across rounds. Given an edited image from the previous round, we cache its intermediate activations as the reference state. In the current round, we first run the model at the initial denoising step and compare deep-layer activations with the cached reference to obtain an editable-region mask (Section 4.2). We then decouple tokens into editable and non-editable sets and apply sparse computation only to editable tokens while keeping dense context through full keys and values. For non-editable tokens, their activations are directly retrieved from the cache and injected back after each Transformer block (Section 4.3). This design reduces redundant computation throughout subsequent denoising steps while preserving global context and cross-round consistency.

4.2. Early Adaptive Region Recognition

To achieve high-efficiency editing without compromising global structural integrity, we propose the **Early Adaptive Region Recognition (EARR)** mechanism. This module leverages the observation that semantic modifications in DiTs are most discernible in the deep layers during the initial denoising stages.

Activation Caching and Similarity Analysis. Building upon the similarity analysis in Section 3, we define a reference activation state from the previous editing round $k - 1$. Specifically, we cache the output activations of the final Transformer layer L at the first denoising step $t = 0$, denoted as $\mathbf{H}_{0,L}^{(k-1)} \in \mathbb{R}^{N \times d}$, where N is the number of tokens and d is the hidden dimension. During the current round k , once $\mathbf{H}_{0,L}^{(k)}$ is computed for the same layer and step, we invoke an *Adaptive Similarity Comparator*. For each token index $i \in \{1, \dots, N\}$, the discrepancy score δ_i is computed as

$$\delta_i = \left\| \mathbf{h}_{i,0,L}^{(k)} - \mathbf{h}_{i,0,L}^{(k-1)} \right\|_2. \quad (2)$$

Tokens with $\delta_i > \tau$ are marked as potential edit candidates, forming an initial sparse binary mask $\mathcal{M}_{\text{raw}} \in \{0, 1\}^N$ defined by $\mathcal{M}_{\text{raw}}(i) = 1$ if $\delta_i > \tau$ and 0 otherwise.

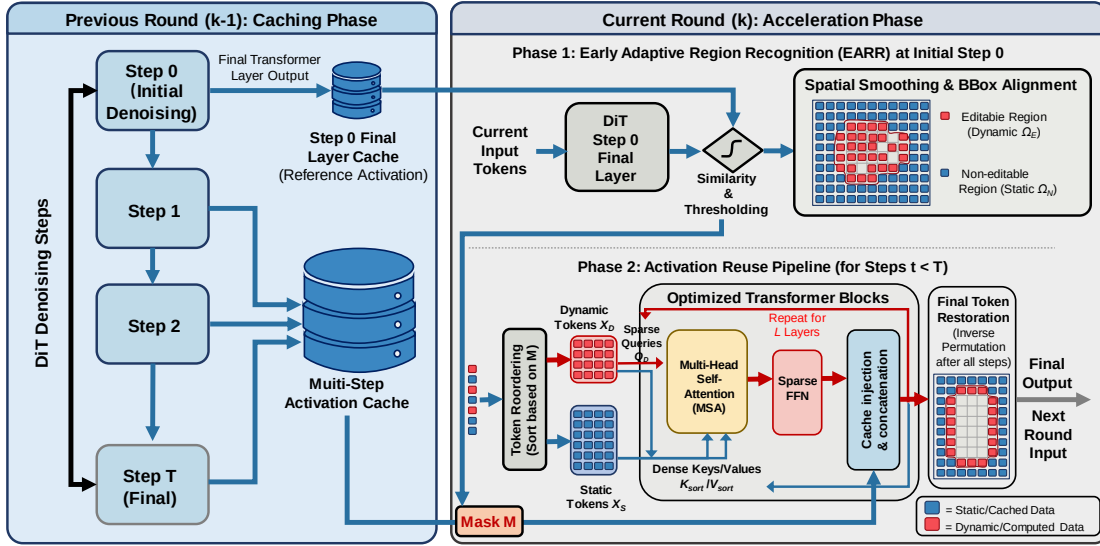


Figure 2. System overview of CacheEdit. We detect editable regions using early-step deep-layer activation differences across rounds, then reuse cached activations for non-editable tokens while recomputing only editable tokens with sparse-query and dense-key/value attention.

Spatial Smoothing and Bounding-box Alignment. To maintain visual consistency and eliminate boundary artifacts between editable and non-editable zones, $\mathcal{M}_{raw}$ undergoes a *Spatial Smoothing Module*. Given that localized edits typically exhibit spatial locality, we apply a morphological dilation function f_{smooth} to expand sparse token indices into a contiguous region:

$$\Omega_E = \{i \mid f_{smooth}(\mathcal{M}_{raw})(i) = 1\}. \quad (3)$$

In practice, we further align the detected region with DiT patch structure by taking the minimal bounding box (BBox) that encloses Ω_E in the 2D token grid, and we mark all tokens inside this BBox as editable. Tokens in Ω_E are designated as the **Editable Region**, and the complement set $\Omega_N = \{1, \dots, N\} \setminus \Omega_E$ is treated as the **Non-editable Region**. This early identification at $t = 0$ enables the system to bypass redundant computations for Ω_N in subsequent denoising steps.

4.3. Activation Reuse in Non-editable Regions

Upon identifying the editable region Ω_E (dynamic) and non-editable region Ω_N (static), we introduce an activation cache strategy that enables fine-grained feature reuse across editing rounds. The core idea is to avoid recomputing intermediate representations for tokens that are not affected by the instruction, thereby reducing redundant computation while preserving global context.

Acache: Spatiotemporal feature persistence. Motivated by the key-value (KV) cache used in large language models (LLMs) to avoid repeated computation on previously processed tokens, we introduce Acache to store and reuse high-dimensional activations in the diffusion backbone. Unlike KV caching in autoregressive decoding, which exploits temporal redundancy over token positions, Acache targets spatiotemporal invariance in multi-round image editing, where large portions of the image remain unchanged across rounds.

Concretely, during the reference round we cache intermediate activations across Transformer layers, and treat the cached activations as the reference state for the next round. In subsequent rounds, for tokens belonging to the static set Ω_N , the model bypasses the standard forward computation and directly injects the corresponding cached activations from Acache. Importantly, Acache stores the full FFN output activation tensors (rather than only attention-related components), so the non-editable background can be preserved with high numerical fidelity.

By shifting a portion of the workload from compute-intensive operations to efficient memory retrieval, Acache reallocates computation to the editable region Ω_E , yielding a meaningful speedup while maintaining consistent representations for the static context.

Reordering-based Sparse Computation Pipeline. For the dynamic tokens in Ω_E , full re-computation is necessary. However, purely sparse computation often breaks the global context required by the attention mechanism. To address

this, we design a pipeline that performs *Sparse Query*, *Dense Key/Value* attention.

The process minimizes indexing overhead by performing token reordering only once per denoising step, rather than per layer. The workflow for a single step is as follows:

1. **Initial Reordering:** At the input of the first transformer layer, we perform a global sort on the token list $\mathbf{X}$. Dynamic tokens $\mathbf{X}_D$ and Static tokens $\mathbf{X}_S$ are grouped and concatenated:

$$\mathbf{X}_{sort} = \text{Concat}(\mathbf{X}[\Omega_E], \mathbf{X}[\Omega_N]) \quad (4)$$

Since DiT self-attention does not use a causal mask, the attention operator is permutation-equivariant. Therefore, reordering does not change the mathematical outcome as long as the corresponding positional embeddings are reordered together with the tokens.

2. **Sparse Query, Dense Key/Value Attention:** Within each transformer block, we optimize the Multi-Head Self-Attention (MSA). We generate Queries (Q) solely from the dynamic segment $\mathbf{X}_D$, while Keys (K) and Values (V) are derived from the full sequence $\mathbf{X}_{sort}$:

$$Q_D = \mathbf{X}_D W_Q, \quad \{K, V\}_{sort} = \mathbf{X}_{sort} \{W_K, W_V\} \quad (5)$$

The attention operation is then:

$$\mathbf{Y}_D = \text{Softmax} \left(\frac{Q_D K_{sort}^T}{\sqrt{d}} \right) V_{sort} \quad (6)$$

This formulation ensures that dynamic tokens can attend to the entire image context (both static and dynamic backgrounds) while reducing the complexity of the attention map from $N \times N$ to $|\Omega_E| \times N$.

3. **Sparse FFN and Cache Injection:** The Feed-Forward Network (FFN) processes only the output $\mathbf{Y}_{attn}$ (corresponding to dynamic tokens). To prepare the input for the *next* transformer layer, we concatenate the newly computed dynamic activations with the cached static activations $\mathcal{C}_l[\Omega_N]$ retrieved from the previous round:

$$\mathbf{X}_{next} = \text{Concat}(\text{FFN}(\mathbf{Y}_{attn}), \mathcal{C}_l[\Omega_N]) \quad (7)$$

This loop continues for all L layers.

4. **Final Restoration:** After the final layer of the current denoising step, we apply an inverse permutation π^{-1} to restore all tokens to their original spatial coordinates, ensuring correct spatial alignment for the subsequent step.

This "Sort-Once, Compute-Sparsely" paradigm significantly reduces the overhead of gathering and scattering tokens at every layer.

4.4. Theoretical Analysis of GFLOPS Reduction

This section quantifies the FLOPs reduction achieved by CacheEdit when using the sparse-query and dense-key/value computation described in Section 4.3. We focus on the dominant costs within a single Transformer block.

Baseline FLOPs of a DiT block. Let N denote the number of tokens and d the hidden dimension. We adopt the standard decomposition into (i) linear projections and (ii) attention matrix multiplications.

Linear projections. A standard block computes Q, K, V, O projections and an FFN with expansion ratio 4. The FLOPs are

$$\mathcal{C}_{lin}^{base} = \underbrace{4Nd^2}_{Q,K,V,O} + \underbrace{8Nd^2}_{FFN} = 12Nd^2. \quad (8)$$

Attention matrix multiplications. Self-attention includes computing attention scores and applying them to values,

$$\mathcal{C}_{attn}^{base} = \underbrace{N^2d}_{QK^T} + \underbrace{N^2d}_{AV} = 2N^2d. \quad (9)$$

Combining the two terms yields the baseline per-block cost

$$\mathcal{G}_{base} = 12Nd^2 + 2N^2d. \quad (10)$$

FLOPs with sparse queries and dense keys and values. Let Ω_E be the editable set and define $m = |\Omega_E|$ and $r = m/N$. CacheEdit recomputes only the dynamic tokens while keeping dense context for attention.

Linear projections. We compute K and V for all tokens to preserve global context, but compute Q , output projection O , and the FFN only for the m dynamic tokens. This gives

$$\mathcal{C}_{lin}^{ours} = \underbrace{2Nd^2}_{K,V} + \underbrace{(1+1+8)md^2}_{Q,O,FFN} = (2+10r)Nd^2. \quad (11)$$

Attention matrix multiplications. Since only m queries are computed, the attention matrix has shape $m \times N$:

$$\mathcal{C}_{attn}^{ours} = \underbrace{mNd}_{QK^T} + \underbrace{mNd}_{AV} = 2rN^2d. \quad (12)$$

Therefore, the per-block cost of CacheEdit is

$$\mathcal{G}_{ours} = (2+10r)Nd^2 + 2rN^2d. \quad (13)$$

Closed-form reduction ratio. The theoretical reduction ratio ρ is

$$\rho(r) = 1 - \frac{\mathcal{G}_{ours}}{\mathcal{G}_{base}} = 1 - \frac{(2+10r)Nd^2 + 2rN^2d}{12Nd^2 + 2N^2d}. \quad (14)$$

Using the aspect ratio $\gamma = N/d$, we obtain the equivalent compact form

$$\rho(r, \gamma) = 1 - \frac{(2+10r) + 2r\gamma}{12 + 2\gamma}. \quad (15)$$

Discussion. This expression is monotone decreasing in r , and it correctly recovers the full-computation baseline when $r = 1$ (that is, $\rho = 0$). In practice, when $r \approx 0$ the model can reuse cached activations for all tokens, which yields near-zero compute for the block. The above analysis instead characterizes the typical case where at least one dynamic token exists and dense K, V are still computed to preserve global context.

5. Evaluation

5.1. Experimental Settings

Model & Dataset. We evaluate CacheEdit on two state-of-the-art open-source instruction-based image editing (IIE) models, FLUX.1 Kontext (Batifol et al., 2025) and Qwen-Image-Edit (Wu et al., 2025). Unless otherwise specified, all methods use a fixed budget of 30 sampling steps. We set the guidance scale to 16 for FLUX.1 Kontext and 4 for Qwen-Image-Edit.

Because the original technical reports for these models primarily evaluate single-round editing, we construct a multi-round benchmark to better reflect interactive usage. We start from 108 base images in KontextBench (Batifol et al., 2025) and use Gemini3-pro (Team et al., 2023) to expand the original single-round instructions into sequences of eight consecutive editing rounds. These sequences encompass a diverse range of task types, including background modification, color alteration, human manipulation, style transfer, subject addition/removal/replacement, and text editing, as shown in Section B. We then run the original unmodified models on these instruction sequences to obtain the full-recomputation baseline trajectories for comparison.

Evaluation Metrics. We evaluate CacheEdit from two perspectives: generation quality and computational efficiency.

Generation quality. We adopt two complementary evaluation protocols. First, we measure the deviation between accelerated outputs and the corresponding full-recomputation outputs using PSNR, SSIM (Wang et al., 2004), and LPIPS (Zhang et al., 2018). These reconstruction metrics quantify how closely CacheEdit matches the baseline trajectory.

However, deviation-based metrics can be misleading in multi-round editing. CacheEdit often improves background stability and instruction adherence relative to full recomputation, in which case a larger deviation may indicate a better edit rather than a degraded one. Therefore, we additionally adopt a VLM-as-a-judge protocol and use GPT-4o (Achiam et al., 2023) to assess overall image quality and instruction alignment, with particular emphasis on cases that show large deviations.

Computational efficiency. We report end-to-end latency and the acceleration ratio (speedup), defined as the ratio

between the baseline latency and the CacheEdit latency.

Baselines. Since relatively few acceleration methods are designed specifically for IIE, we use RegionE(Chen et al., 2025b) as the primary IIE-oriented baseline. In addition, following common practice in prior acceleration studies, we compare against representative techniques originally proposed for general diffusion models. For timestep redundancy, we include StepSkip (larger step jumps), FORA (block-level caching) (Selvaraju et al., 2024), and Δ -DiT (Chen et al., 2024) and TeaCache (residual-based caching) (Liu et al., 2025a). For spatial redundancy, we include RAS (Liu et al., 2025e) and ToCa (Zou et al., 2024b), which reduce computation by selectively updating tokens.

Implementation Details. We implement the CacheEdit system based on the Hugging Face Diffusers ecosystem for both evaluation models, incorporating a specialized Adaptive Activation Cache (Acache) to manage reusable data across multiple editing rounds. All performance benchmarks are conducted on NVIDIA A100 GPUs. The models operate with a fixed budget of 30 sampling steps. During the initial step ($s = 0$), the system generates a spatial mask by computing the cosine similarity between the output activations of the final Transformer layer from the current and previous rounds. We adopt a similarity threshold of 0.97 to distinguish between edited and non-edited regions, ensuring precise identification of invariant tokens for subsequent activation reuse.

5.2. Experimental Results Analysis

Deviation Analysis from the Base Model. As shown in Table 1, CacheEdit achieves consistently stronger reconstruction fidelity than the competing baselines on both models. For FLUX.1 Kontext, CacheEdit attains a PSNR of 39.112 and an LPIPS of 0.115, while the strongest baseline (RegionE) reaches a PSNR of 29.103 and an LPIPS of 0.182. For Qwen-Image-Edit, CacheEdit also improves reconstruction fidelity, achieving a PSNR of 32.126 and an LPIPS of 0.063, compared with RegionE’s PSNR of 27.315 and LPIPS of 0.098. Overall, these gaps indicate that token-wise reuse preserves non-edited regions more faithfully and avoids accumulating unnecessary changes across rounds.

Assessing Editing Quality via LLM-as-a-Judge. The VLM-based evaluation via GPT-4o further validates the qualitative advantages of our framework. On FLUX.1 Kontext, CacheEdit improves the overall score (G-O) from the vanilla baseline’s 6.336 to 6.814. Most notably, our system outperforms the original full-computation model in Semantic Consistency (G-SC), scoring 7.816 versus the vanilla 7.566. This "Better than Vanilla" phenomenon is a critical finding of our study. In a multi-round setting, the original model often fails to anchor static background elements, leading to unintended semantic shifts as the editing chain

Table 1. Quantitative comparison on the multi-round IIE benchmark: reconstruction metrics (PSNR/SSIM/LPIPS) against full recomputation, GPT-4o judge scores, and end-to-end latency/speedup measured on a single NVIDIA A100 GPU.

Model	Against Vanilla			GPT-4o Score			Efficiency	
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	G-SC $\uparrow$	G-PQ $\uparrow$	G-O $\uparrow$	Latency (s) $\downarrow$	Speedup $\uparrow$
FLUX.1 Kontext	-	-	-	7.566	6.663	6.336	35.159	1.000
+ Stepskip	26.812	0.636	0.174	6.833	6.584	6.172	27.382	1.284
+ FORA	24.384	0.639	0.196	6.794	6.483	6.076	24.115	1.458
+ Δ -DiT	21.723	0.568	0.308	6.775	6.565	6.122	21.716	1.619
+ TeaCache	29.002	0.659	0.157	6.936	6.496	6.164	22.02	1.597
+ RAS	26.609	0.621	0.185	6.920	6.438	6.169	29.307	1.200
+ ToCa	22.986	0.597	0.237	6.703	6.252	5.957	27.066	1.299
+ RegionE	29.103	0.655	0.182	6.986	6.599	6.245	21.645	1.624
+ CacheEdit	39.112	0.925	0.115	7.816	6.752	6.814	18.359	1.915
Qwen-Image-Edit	-	-	-	8.077	7.550	7.384	66.043	1.000
+ Stepskip	24.941	0.713	0.164	7.928	7.481	7.261	52.332	1.262
+ FORA	21.942	0.690	0.208	7.871	7.372	7.193	52.260	1.24
+ Δ -DiT	25.020	0.656	0.247	7.804	7.332	7.112	52.084	1.268
+ TeaCache	24.831	0.720	0.159	7.922	7.448	7.252	49.029	1.347
+ RAS	23.899	0.703	0.191	7.988	7.296	7.206	55.082	1.199
+ ToCa	OOM	OOM	OOM	OOM	OOM	OOM	OOM	OOM
+ RegionE	27.315	0.855	0.098	8.085	7.589	7.415	46.620	1.417
+ CacheEdit	32.126	0.926	0.063	8.358	7.636	7.889	40.944	1.613

Table 2. Speedup of **CacheEdit** across different editing types on the multi-round IIE benchmark.

Editing Type	FLUX.1 Kontext	Qwen-Image-Edit
Background Change	1.334	1.157
Color Alteration	2.287	1.739
Human Manipulation	2.064	1.684
Style Transfer	1.156	1.137
Subject Edit	2.793	2.376
Text Editing	1.752	1.585
Average	1.915	1.613

grows longer. By freezing the activations of invariant tokens, CacheEdit provides a form of implicit structural grounding that enhances the model’s focus on the edited regions. On Qwen-Image-Edit, CacheEdit likewise improves the GPT-4o overall score (G-O) from 7.384 to 7.889 and increases semantic consistency (G-SC) from 8.077 to 8.358, confirming that sparse computation can serve as a regularizer for long-term consistency.

Efficiency and Speedup Analysis. CacheEdit also delivers substantial end-to-end acceleration. On FLUX.1 Kontext, it reduces latency from 35.159s to 18.359s, corresponding to a $1.915\times$ speedup, outperforming RegionE ($1.624\times$) and Δ -DiT ($1.619\times$). On Qwen-Image-Edit, CacheEdit achieves a $1.613\times$ speedup by reducing latency from 66.043s to 40.944s. Additionally, Table 2 presents the speedup achieved by CacheEdit across different editing tasks.

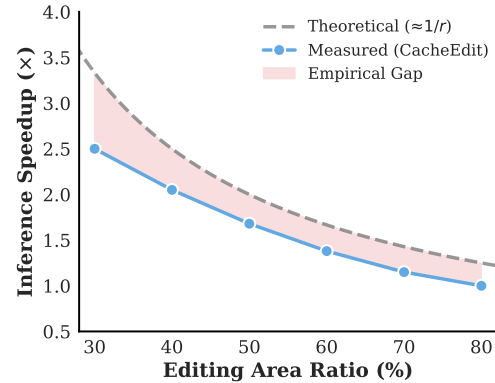


Figure 3. Inference speedup of CacheEdit relative to the editing area ratio.

The efficiency of our framework is intrinsically linked to the spatial sparsity of the editing instructions. As illustrated in Figure 3, the achieved speedup is consistent with the theoretical FLOPs reduction derived in Section 4.4, which depends on the editable ratio r (and the aspect ratio $\gamma = N/d$). In the attention-dominated regime, this prediction simplifies to an approximate $1/r$ trend; in practice, the realized speedup is modulated by system factors, including the relative cost of attention/FFN computation versus cache retrieval, as well as sorting and indexing overheads. When edits are localized (e.g., 30% of the image area), CacheEdit recovers nearly 75% of its theoretical potential, while the remaining gap is primarily attributable to dynamic indexing and activa-

tion I/O from Acache. Despite these overheads, eliminating redundant computation from the very first denoising step makes CacheEdit well suited for real-time interactive editing where responsiveness is paramount.

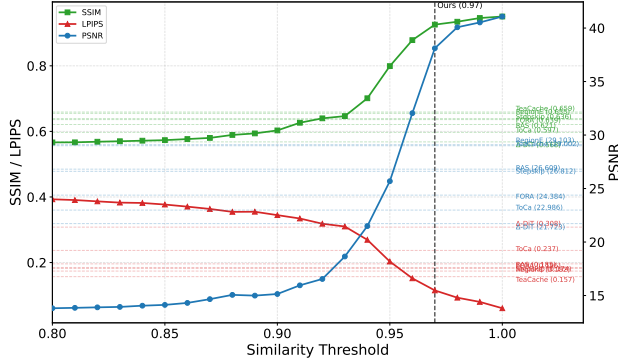


Figure 4. Ablation of the similarity threshold τ on FLUX.1 Kontext. A larger threshold leads to more conservative reuse and improves reconstruction fidelity, while a smaller threshold enables more aggressive reuse. The default setting $\tau = 0.97$ provides a favorable trade-off between fidelity and acceleration.

5.3. Ablation Study

Effect of similarity threshold. The similarity threshold τ controls the trade-off between reuse aggressiveness and output fidelity. A lower threshold marks more tokens as editable and therefore recomputes a larger region, which improves fidelity but reduces the potential speedup. In contrast, a higher threshold increases the number of reused tokens and leads to stronger acceleration, but overly aggressive reuse may introduce visible mismatch when edited tokens are incorrectly classified as static. As shown in Figure 4, the reconstruction quality improves steadily as τ increases from 0.80 to 1.00. Around our default setting $\tau = 0.97$, CacheEdit achieves a favorable balance: it maintains high reconstruction fidelity while still preserving a substantial amount of reusable static computation. This justifies the threshold used in our main experiments.

Breakdown by editing type. Table 2 reports the speedup of CacheEdit across different editing types. The results show a clear pattern: broader edits, such as background modification and style transfer, lead to smaller speedups because a larger portion of the image must be recomputed. In contrast, localized edits, such as subject addition, removal, or replacement, benefit more from token-wise reuse and achieve much higher acceleration. This trend is consistent with the theoretical analysis in Section 4.4, where the FLOPs reduction depends directly on the editable-token ratio r .

6. Limitations and Future Works

The main limitation of CacheEdit is its additional memory overhead. Acache stores intermediate activation tensors for reuse across editing rounds, which makes CacheEdit an explicit memory-for-latency trade-off. In our current implementation, although we cache activations for only five denoising steps per round, the additional memory overhead on FLUX.1 Kontext is still 31.36 GB. This cost may limit deployment on memory-constrained GPUs, especially for higher resolutions, larger batch sizes, or multiple concurrent editing sessions.

This limitation also points to an important future direction. Our analysis shows that cached activations in non-edited regions are highly similar across adjacent rounds, denoising steps, and Transformer layers, suggesting substantial redundancy in Acache. Future work can exploit this redundancy through activation compression, such as low-bit quantization, low-rank approximation, delta encoding, or selective layer/step caching. In addition, the practical speedup of CacheEdit is affected by cache I/O, token reordering, and cache injection. Designing an efficient pipeline that overlaps cache loading, decompression, and sparse computation could hide part of this overhead and make activation reuse more practical for real-world interactive editing systems.

7. Conclusion

We presented CacheEdit, a training-free framework that accelerates multi-round instruction-based image editing via token-wise activation reuse across rounds. Guided by a systematic similarity analysis, we found that latent-space differences between consecutive rounds become pronounced only in the late denoising stages, whereas deep-layer activations at the first denoising step already provide a reliable signal for localizing edited regions. Building on this finding, CacheEdit performs Early Adaptive Region Recognition to identify editable tokens, and then accelerates inference by recomputing only the editable region while injecting cached activations for non-editable tokens through a sparse-query, dense-key/value pipeline. Experiments on FLUX.1 Kontext and Qwen-Image-Edit show that CacheEdit delivers substantial end-to-end speedups while preserving, and often improving, instruction following and cross-round structural consistency. These results indicate that multi-round editing contains significant inter-round redundancy that can be exploited without model retraining.

Acknowledgments

This paper is supported in part by the Chinese Academy of Sciences under grant No. XDB0660103. The corresponding authors are Ying Wang, Huawei Li and Xiaowei Li.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Batifol, S., Blattmann, A., Boesel, F., Consul, S., Diagne, C., Dockhorn, T., English, J., English, Z., Esser, P., Kulal, S., et al. Flux. 1 kontext: Flow matching for in-context image generation and editing in latent space. *arXiv e-prints*, pp. arXiv–2506, 2025.
- Cao, P., Zhou, F., Song, Q., and Yang, L. Controllable generation with text-to-image diffusion models: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- Castells, T., Song, H.-K., Kim, B.-K., and Choi, S. Ld-pruner: Efficient pruning of latent diffusion models using task-agnostic insights. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 821–830, 2024.
- Chen, L., Meng, Y., Tang, C., Ma, X., Jiang, J., Wang, X., Wang, Z., and Zhu, W. Q-dit: Accurate post-training quantization for diffusion transformers. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 28306–28315, 2025a.
- Chen, P., Shen, M., Ye, P., Cao, J., Tu, C., Bouganis, C.-S., Zhao, Y., and Chen, T. δ -dit: A training-free acceleration method tailored for diffusion transformers. *arXiv preprint arXiv:2406.01125*, 2024.
- Chen, P., Zeng, X., Zhao, M., Shen, M., Ye, P., Xiang, B., Wang, Z., Cheng, W., Yu, G., and Chen, T. Regione: Adaptive region-aware generation for efficient image editing. *arXiv preprint arXiv:2510.25590*, 2025b.
- Chen, R., Mills, K. G., Jiang, L., Gao, C., and Niu, D. Re-ttention: Ultra sparse visual generation via attention statistical reshape. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025c.
- Chu, H., Wu, W., Feng, G., and Zhang, Y. Omnicache: A trajectory-oriented global perspective on training-free cache reuse for diffusion transformer models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 16302–16312, 2025.
- Croitoru, F.-A., Hondru, V., Ionescu, R. T., and Shah, M. Diffusion models in vision: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 45(9): 10850–10869, 2023.
- Guo, B., Tang, S., Zeng, C., and Shen, Z. Mosaicediff: Training-free structural pruning for diffusion model acceleration reflecting pretraining dynamics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1655–1664, 2025.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- Huang, Y., Huang, J., Liu, Y., Yan, M., Lv, J., Liu, J., Xiong, W., Zhang, H., Cao, L., and Chen, S. Diffusion model-based image editing: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.
- Jiang, X., Li, S., Yang, L., Feng, T., Di, Z., Lu, W., Zhu, G., Lin, X., Liu, K., Yu, Y., Lan, T., Yang, G., Qu, L., Zhang, L., and Wang, W. Instgenie: Generative image editing made efficient with mask-aware caching and scheduling, 2025. URL <https://arxiv.org/abs/2505.20600>.
- Kawar, B., Zada, S., Lang, O., Tov, O., Chang, H., Dekel, T., Mosseri, I., and Irani, M. Imagic: Text-based real image editing with diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 6007–6017, 2023.
- Kim, B.-K., Song, H.-K., Castells, T., and Choi, S. Bk-sdm: Architecturally compressed stable diffusion for efficient text-to-image generation. In *Workshop on Efficient Systems for Foundation Models@ ICML2023*, 2023.
- Li, X., Liu, Y., Lian, L., Yang, H., Dong, Z., Kang, D., Zhang, S., and Keutzer, K. Q-diffusion: Quantizing diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 17535–17545, 2023.
- Liu, F., Zhang, S., Wang, X., Wei, Y., Qiu, H., Zhao, Y., Zhang, Y., Ye, Q., and Wan, F. Timestep embedding tells: It’s time to cache for video diffusion model. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 7353–7363, 2025a.
- Liu, L., Wang, Y., Zhao, X., Chen, W., Li, H., Li, X., and Han, Y. An automatic neural network architecture-and-quantization joint optimization framework for efficient model inference. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(5):1497–1510, 2023.

- Liu, L., Xu, Z., He, Y., Wang, Y., Li, H., Li, X., and Han, Y. Drift: Leveraging distribution-based dynamic precision quantization for efficient deep neural network acceleration. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pp. 1–6, 2024.
- Liu, L., Cheng, L., Ren, H., Xu, Z., Pan, Y., Wang, M., Li, X., Han, Y., and Wang, Y. Comet: Towards practical w4a4kv4 llms serving. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pp. 131–146, 2025b.
- Liu, L., Zhao, S., Li, B., Ren, H., Xu, Z., Wang, M., Li, X., Han, Y., and Wang, Y. Make llm inference affordable to everyone: Augmenting gpu memory with ndp-dimm. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 1751–1765. IEEE, 2025c.
- Liu, L., Zhao, X., Li, G., Li, D., Wang, M., Han, Y., Li, X., et al. Bawa: Automatic optimizing pruning metric for large language models with balanced weight and activation. In *Forty-second International Conference on Machine Learning*, 2025d.
- Liu, Z., Yang, Y., Zhang, C., Zhang, Y., Qiu, L., You, Y., and Yang, Y. Region-adaptive sampling for diffusion transformers. *arXiv preprint arXiv:2502.10389*, 2025e.
- Ma, X., Fang, G., Bi Mi, M., and Wang, X. Learning-to-cache: Accelerating diffusion transformer via layer caching. *Advances in Neural Information Processing Systems*, 37:133282–133304, 2024a.
- Ma, X., Fang, G., and Wang, X. Deepcache: Accelerating diffusion models for free. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 15762–15772, 2024b.
- Ma, X., Yu, R., Fang, G., and Wang, X. dkv-cache: The cache for diffusion language models. *arXiv preprint arXiv:2505.15781*, 2025.
- Nguyen, T. H. and Tran, A. Swiftbrush: One-step text-to-image diffusion model with variational score distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7807–7816, 2024.
- Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 4195–4205, 2023.
- Qiao, L., Dai, Y., Huang, Y., Kan, H., Shi, J., and An, H. Flashomni: A unified sparse attention engine for diffusion transformers. *arXiv preprint arXiv:2509.25401*, 2025.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PmLR, 2021.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10684–10695, 2022.
- Sauer, A., Lorenz, D., Blattmann, A., and Rombach, R. Adversarial diffusion distillation. In *European Conference on Computer Vision*, pp. 87–103. Springer, 2024.
- Selvaraju, P., Ding, T., Chen, T., Zharkov, I., and Liang, L. Fora: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint arXiv:2407.01425*, 2024.
- So, J., Lee, J., Ahn, D., Kim, H., and Park, E. Temporal dynamic quantization for diffusion models. *Advances in neural information processing systems*, 36:48686–48698, 2023.
- Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Wang, Z., Bovik, A. C., Sheikh, H. R., and Simoncelli, E. P. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004.
- Wimbauer, F., Wu, B., Schoenfeld, E., Dai, X., Hou, J., He, Z., Sanakoyeu, A., Zhang, P., Tsai, S., Kohler, J., et al. Cache me if you can: Accelerating diffusion models through block caching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6211–6220, 2024.
- Wu, C., Li, J., Zhou, J., Lin, J., Gao, K., Yan, K., Yin, S.-m., Bai, S., Xu, X., Chen, Y., et al. Qwen-image technical report. *arXiv preprint arXiv:2508.02324*, 2025.
- Xiang, Q., Zhang, M., Shang, Y., Wu, J., Yan, Y., and Nie, L. Dkdm: Data-free knowledge distillation for diffusion models with any architecture. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 2955–2965, 2025.
- Yuan, Z., Zhang, H., Pu, L., Ning, X., Zhang, L., Zhao, T., Yan, S., Dai, G., and Wang, Y. Ditfastattn: Attention compression for diffusion transformer models. *Advances in Neural Information Processing Systems*, 37:1196–1219, 2024.

- Zhang, R., Isola, P., Efros, A. A., Shechtman, E., and Wang, O. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 586–595, 2018.
- Zhao, M., Chen, P., Yu, C., Wen, Y., Tan, X., and Chen, T. Pioneering 4-bit fp quantization for diffusion models: Mixup-sign quantization and timestep-aware fine-tuning. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 18134–18143, 2025.
- Zhao, X., Wang, Y., Cai, X., Liu, C., and Zhang, L. Linear symmetric quantization of neural networks for low-precision integer hardware. In *International Conference on Learning Representations*, 2020.
- Zou, C., Liu, X., Liu, T., Huang, S., and Zhang, L. Accelerating diffusion transformers with token-wise feature caching. *arXiv preprint arXiv:2410.05317*, 2024a.
- Zou, C., Zhang, E., Guo, R., Xu, H., He, C., Hu, X., and Zhang, L. Accelerating diffusion transformers with dual feature caching. *arXiv preprint arXiv:2412.18911*, 2024b.

CacheEdit: Efficient Multi-round Image Editing via Adaptive Token-wise Reuse

Supplementary Material

We organize the supplementary material as follows:

- Section A: Pseudocode of CacheEdit
- Section B: Visualization Results in The Benchmark
- Section C: Detailed Similarity Analysis

A. Pseudocode of CacheEdit

Algorithm 1 CacheEdit: Efficient Multi-round Image Editing via Adaptive Token-wise Reuse.

Input: Initial noise $\mathbf{z}_T^{(k)}$; Reference cache $\mathcal{C}^{(k-1)}$; Threshold τ ; Steps T

Output: Edited image $\mathbf{z}_0^{(k)}$

```

1  Phase 1: Early Adaptive Region Recognition (EARR) // at  $t = 0$ 
   Compute final layer activations  $\mathbf{H}_{0,L}^{(k)}$  via full forward pass
2  for each token  $i \in \{1, \dots, N\}$  do
3       $\delta_i \leftarrow \left\| \mathbf{h}_{i,0,L}^{(k)} - \mathbf{h}_{i,0,L}^{(k-1)} \right\|_2$  // Compute discrepancy
       $\mathcal{M}_{\text{raw}}(i) \leftarrow \mathbf{1}[\delta_i > \tau]$ 
4   $\Omega_E \leftarrow \text{BBox}(\text{Smooth}(\mathcal{M}_{\text{raw}}))$ ;  $\Omega_N \leftarrow \{1, \dots, N\} \setminus \Omega_E$ 
5
   Phase 2: Sparse Computation & Cache Injection // for  $t > 0$ 
   for  $t = 1$  to  $T$  do
6        $\mathbf{X}_{\text{sort}} \leftarrow \text{Concat}(\mathbf{X}[\Omega_E], \mathbf{X}[\Omega_N])$  // Global reordering once per step
7       for layer  $l = 1$  to  $L$  do
8            $\mathbf{X}_D \leftarrow \mathbf{X}_{\text{sort}}[0 : |\Omega_E|]$  // Extract Dynamic tokens
           // Sparse Query, Dense Context Attention
            $Q_D \leftarrow \mathbf{X}_D W_Q^l$ ,  $\{K, V\}_{\text{sort}} \leftarrow \mathbf{X}_{\text{sort}}\{W_K^l, W_V^l\}$   $\mathbf{Y}_D \leftarrow \text{Softmax}(Q_D K_{\text{sort}}^\top / \sqrt{d}) V_{\text{sort}}$ 
           // FFN & Dynamic-Static Integration
10           $\mathbf{Y}_{\text{FFN}} \leftarrow \text{FFN}(\mathbf{Y}_D)$   $\mathbf{X}_{\text{sort}} \leftarrow \text{Concat}(\mathbf{Y}_{\text{FFN}}, \mathcal{C}_{l,t}^{(k-1)}[\Omega_N])$  // Inject cached static features
11       $\mathbf{X}_{\text{out}} \leftarrow \text{InversePermute}(\mathbf{X}_{\text{sort}})$  // Restore spatial layout
      Update latent  $\mathbf{z}_{T-t}^{(k)}$ 
12 return  $\mathbf{z}_0^{(k)}$ 

```

B. Visualization Results in The Benchmark

As shown in Figure 5, Figure 6, and Figure 7 present several experimental cases from our evaluation on the Flux model. Each initial image underwent eight consecutive rounds of instruction-based editing, covering a diverse range of tasks including background modification, color alteration, human manipulation, style transfer, subject addition/removal/replacement, and text editing. As illustrated, the images generated by CacheEdit are significantly more consistent with the original model compared to the primary baseline, RegionE. In certain cases, CacheEdit even achieves instruction-based edits that the original model fails to execute. Examples include the instruction "add a street lamp illuminating the wall" in Round 7 (Figure 5, right) and "change the name to 'Anna'" in Round 4 (Figure 7, right).

C. Detailed Similarity Analysis

As shown in Figure 8 illustrates the similarity of image latents across all corresponding steps in two consecutive editing rounds. It is evident that the denoising process is non-linear; image modifications remain subtle during the early steps, with significant changes only emerging in the middle and late stages. Furthermore, the non-editable regions exhibit remarkably high similarity between adjacent rounds, where most tokens are nearly identical, offering substantial potential for computational reuse.

As shown in Figure 9 presents the output activation similarity for all Transformer layers at the initial denoising step across consecutive rounds. In each subplot, the left and right halves represent the tokens of the generated and reference images, respectively. Notably, the activations in the editable region of the reference image undergo a drastic shift as early as the first layer, indicating that the model’s initial layer is highly sensitive to the editing instructions. In subsequent layers, these modifications gradually propagate to the generated image. While the activations in the editable region of the generated image show a distinct change by the final layer, this variation remains subtle when mapped back to the image latent due to the relatively small weights assigned to early denoising steps. Consequently, we leverage the output activations of the Transformer blocks to distinguish between editable and non-editable regions.

As shown in Figure 10 illustrates the similarity between the image latents at the initial step and all subsequent steps within a single editing round. It is observed that during the early steps, it is difficult to accurately distinguish between edited and non-edited regions. Furthermore, as the denoising process progresses, both regions undergo significant transformations. Combined with our previous cross-round similarity analysis, these results indicate that non-edited regions follow identical evolutionary trajectories across different rounds, further validating the feasibility of activation reuse between consecutive editing iterations.

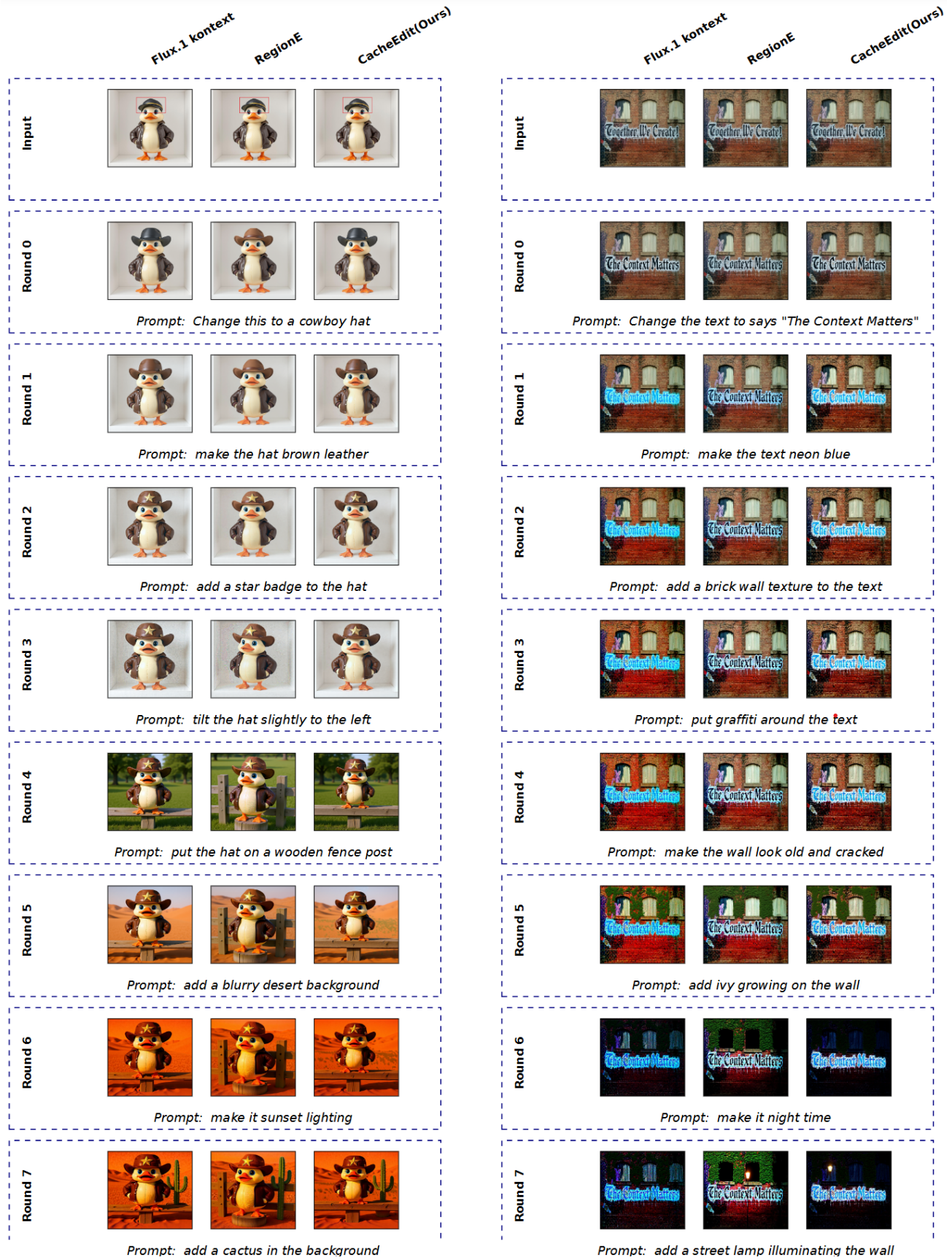


Figure 5. Examples of edited images by CacheEdit and baselines on the Flux model. The figure showcases eight rounds of sequential editing across various task types.



Figure 6. Examples of edited images by CacheEdit and baselines on the Flux model. The figure showcases eight rounds of sequential editing across various task types.

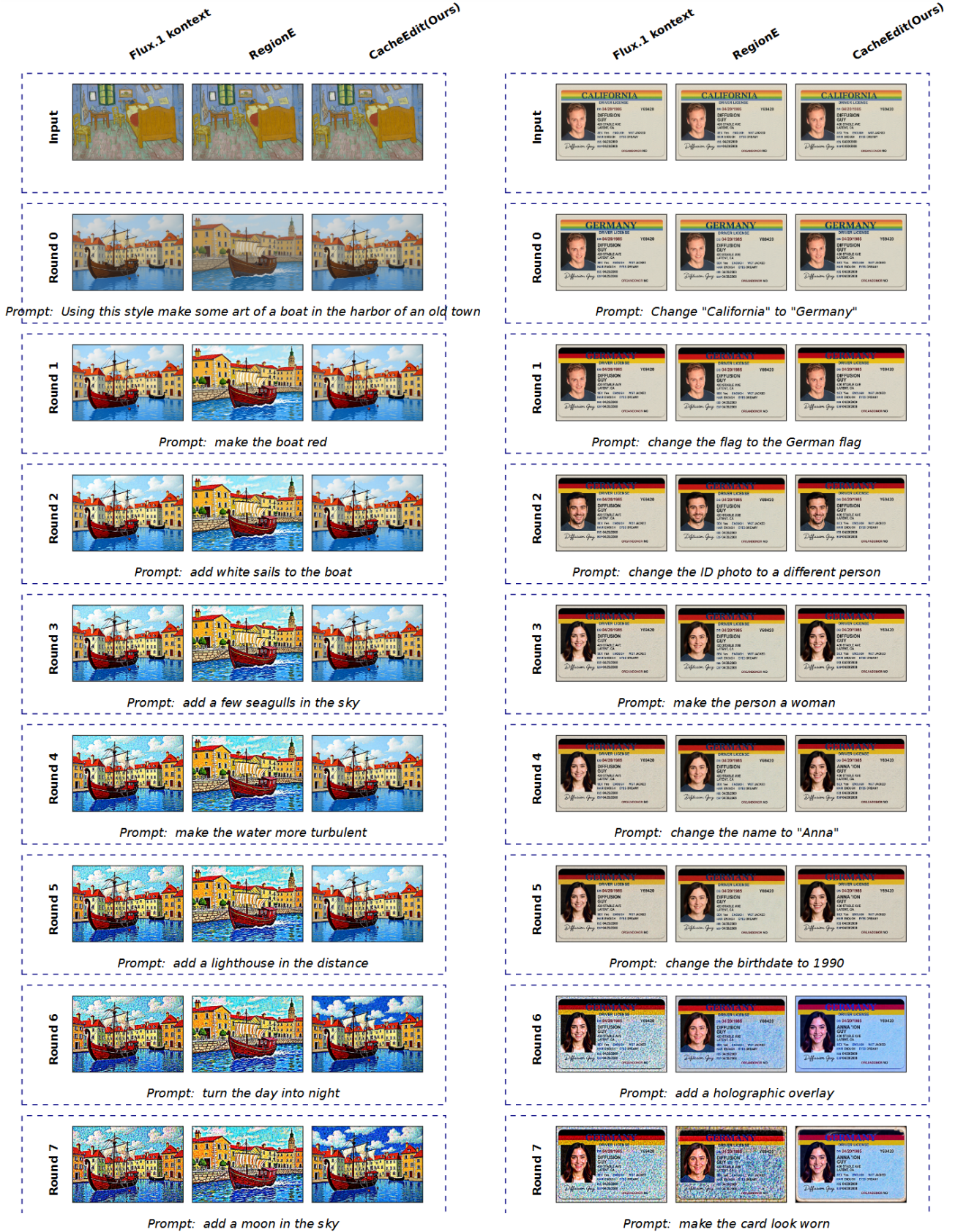


Figure 7. Examples of edited images by CacheEdit and baselines on the Flux model. The figure showcases eight rounds of sequential editing across various task types.

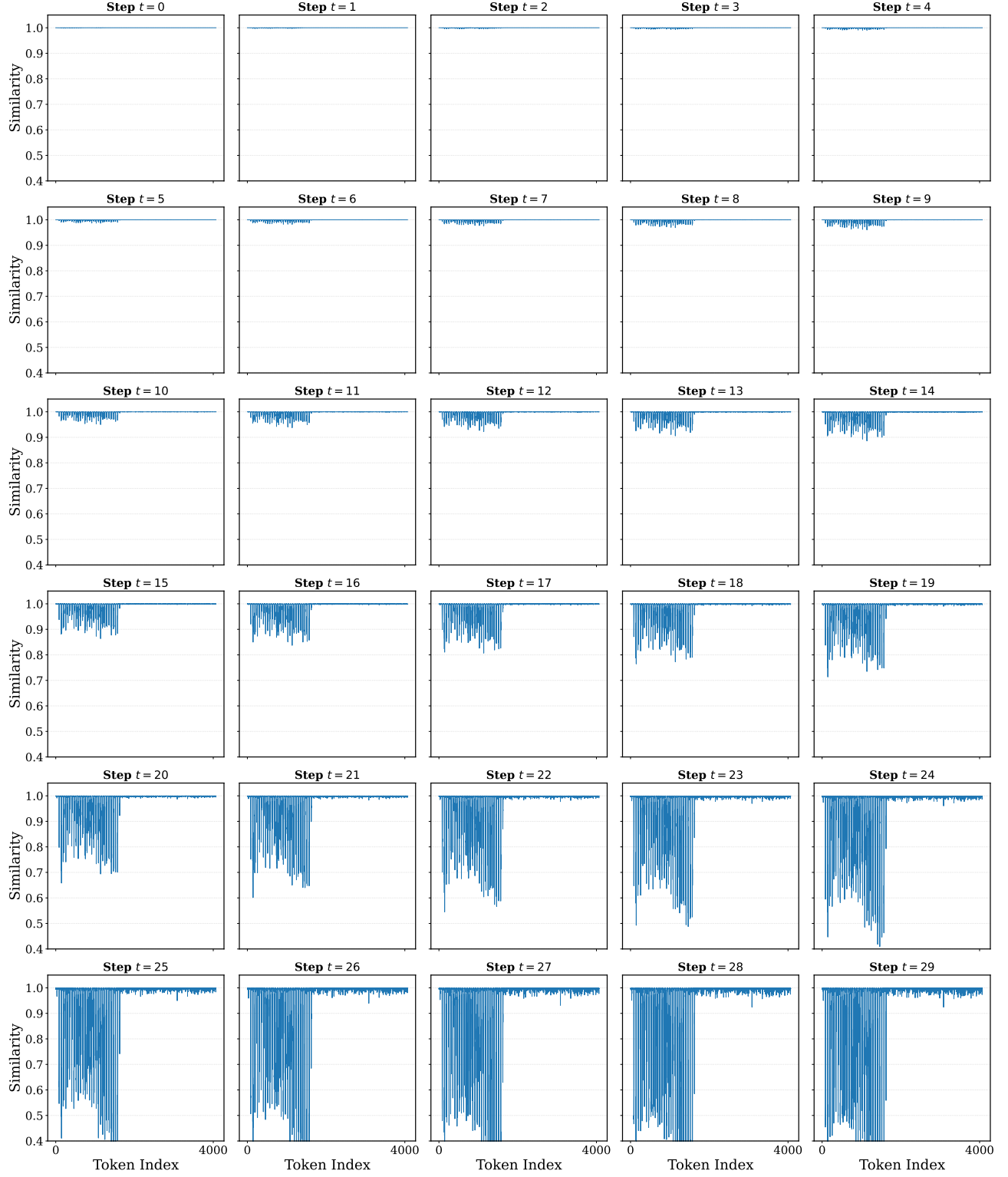


Figure 8. Similarity of image latents across all corresponding steps between adjacent rounds.

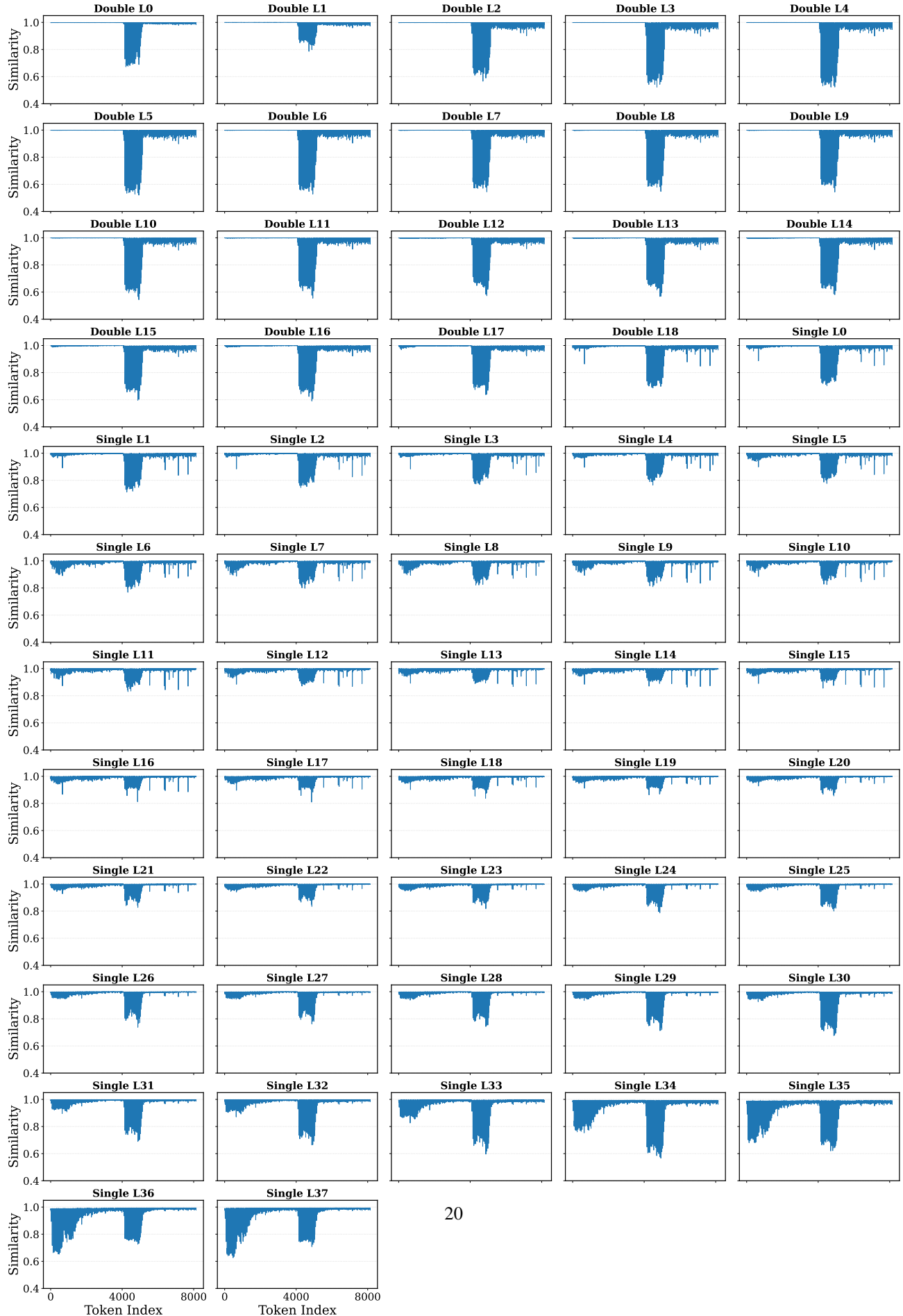


Image Latent Similarity in Single Round at step level (Reference: Step 0)

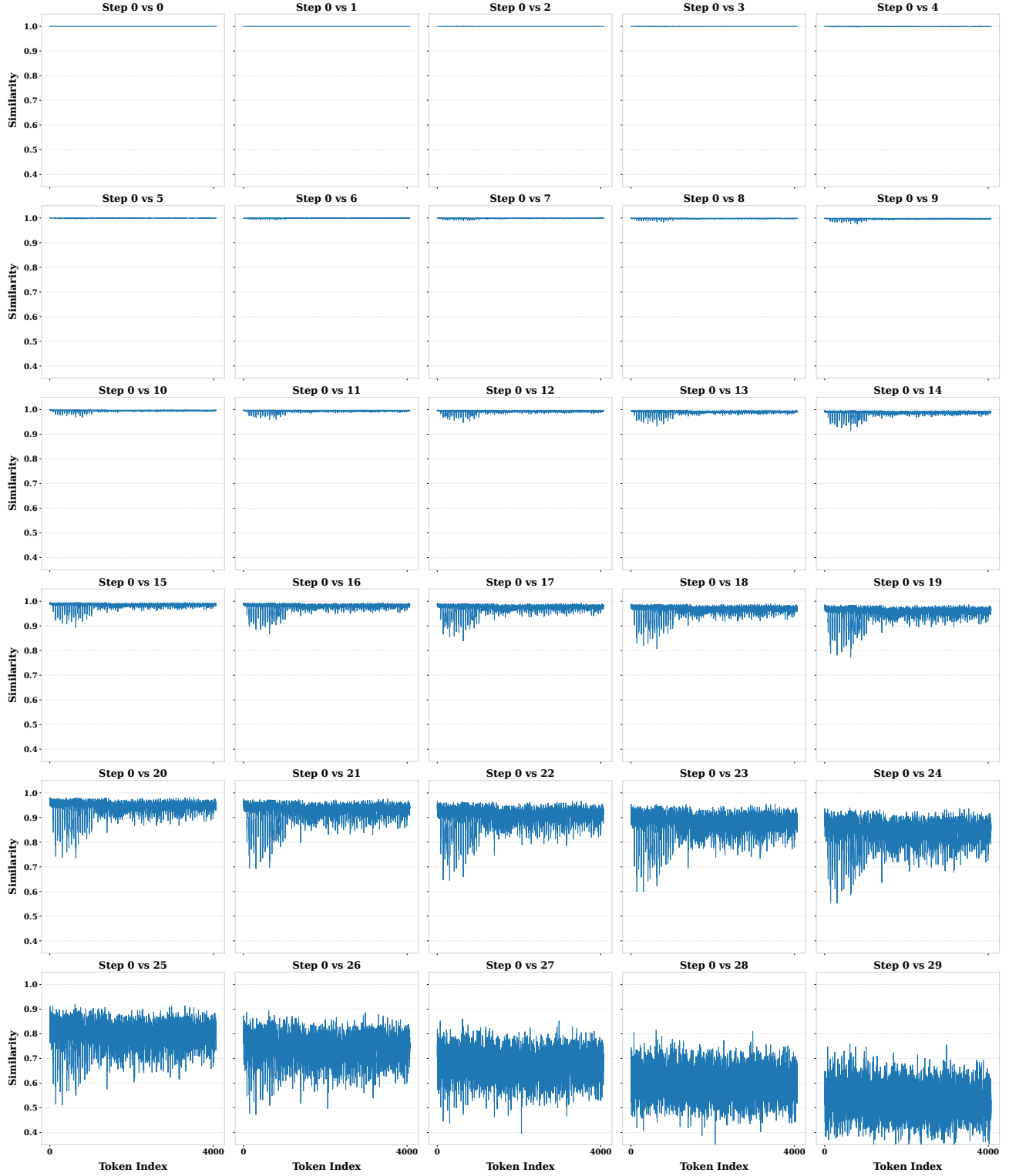


Figure 10. Similarity between the image latent of the initial step and all subsequent steps within a single round.